

MAHATMA GLOBAL GATEWAY TASKFLOW

Native OS Notification & Calendar Sync System documentation

This document provides a comprehensive blueprint of the native Windows OS integration features built into the Mahatma Global Gateway TaskFlow platform, detailing the installation instructions, system workflow, and technical architecture.

1. Core System Architecture

The OS integration bridges the web-application runtime with the native Windows operating system by orchestrating communication between three independent layers:

- **Web Frontend (React/Vite):** Running on port `8080` . Provides settings, configuration triggers, and user-initiated sync handlers.
- **Application Backend (Express/Prisma/SQLite):** Running on port `4000` . Manages task assignments, updates notification flags in the database, generates iCalendar files, and triggers elevated system commands.
- **Native Windows Subsystem (Task Scheduler & WinRT):** Runs strictly on the client's local computer. Executes PowerShell script scripts to fetch status updates and display native system alerts.

2. Detailed Notification System Workflow

The diagram below outlines the sequential lifecycle of task notifications, explaining how a web action triggers a native Windows notification:

Phase 1: Task Assignment & Data Generation

When a Manager assigns a task to a Staff Member, the backend Express server writes a task record and generates a `Notification` database entry. By default, this notification is flagged as `read = false` and `notifiedOnStartup = false` .

Phase 2: One-Click Installation & Local Binding

When the user clicks the **Register OS Startup Alerts** button in the browser Settings:

1. The frontend triggers the authenticated endpoint `POST /api/users/setup-notifier` .

2. The backend writes the user's specific email address to a local configuration file named `mgg_config.json` .
3. The backend executes the PowerShell installer script (`install_notifier_task.ps1`) with elevated Admin privilege (UAC prompt), which registers a logon task named `MGGStartupNotifier` directly inside the Windows Task Scheduler.

Phase 3: Startup Trigger & WinRT Execution

When the user boots their Windows computer and logs into their user account:

1. The Windows Task Scheduler waits for 30 seconds (settling period) and executes the background script `mgg_startup_notifier.ps1` silently in hidden mode.
2. The script reads the local user email out of `mgg_config.json` and queries the backend endpoint `GET /api/notifications/startup?userId=<email>` .
3. The backend returns all unread notifications where `notifiedOnStartup = false` , and immediately updates those database flags to `true` .
4. The PowerShell script receives the JSON response and triggers native Windows Forms notification balloons to pop up on the desktop.

3. Staff Member Setup Instructions

Step A: Setup Windows Startup Alerts

1. Log into your account at `http://localhost:8080` .
2. Click the **Settings** gear icon in the top-right corner.
3. Go to the **Notifications** tab.
4. Scroll to **Windows OS Startup Notifications** and click **Register OS Startup Alerts**.
5. Click **Yes** on the Windows Administrator prompt that appears.

Step B: Sync Deadlines with Windows Calendar

1. Navigate to the **Calendar Sync** tab in Settings.
2. Click the **Sync Automatically** button.
3. Click the downloaded `tasks.ics` file in your browser. Outlook or Windows Calendar will launch and link all your task deadlines.

4. System Maintenance & Troubleshooting

- **Local Server Dependency:** The backend server must be running (`npm run dev`) in order for the background PowerShell alerts to successfully query task deadlines. If the server is offline, the script will silently fail to avoid disrupting the user.
- **Registry Manual Trigger:** If a user needs to verify that the scheduled task is running, they can open PowerShell as Administrator and execute:

```
Start-ScheduledTask -TaskName "MGGStartupNotifier"
```